Code

Brain MRI Image Segmentation Using K-Means Clustering

02 — K-Means Segmentation of the Healthy MRI

Keman Xiang | PhD Application Portfolio

- 1 Project Overview
  - 1.1 Background and motivation
  - 1.2 Research questions
  - 1.3 Analysis roadmap
  - 1.4 Data sources and provenance
- 2 Data Import and Integrity Checks
  - 2.1 Locate the original CSV files
  - 2.2 Read matrices and verify intensity values
- 3 MRI Representation and Computational Rationale
  - 3.1 Visualize the healthy MRI
  - 3.2 Why K-Means rather than a full hierarchical distance matrix?
- 4 Methodology: Five-Cluster K-Means
  - 4.1 Objective function and assumptions
  - 4.2 Fit the course baseline (k = 5)
- 5 Cluster Characteristics and Diagnostics
  - 5.1 Centroids, sizes, and dispersion
  - 5.2 Cluster sizes and within-cluster variation
  - 5.3 Verify centroid and WCSS calculations
- 6 Visualizing the Segmented MRI
  - 6.1 Construct the cluster-label image
  - 6.2 Intensity distribution and centroid positions
- 7 Quantifying Segmentation Error
  - 7.1 Pixel-level grayscale reconstruction error
- 8 Discussion
  - 8.1 What the model demonstrates
  - 8.2 Important limitations
- 9 Conclusion and Next Steps
- 10 Reproducibility
- 11 References

1 Project Overview

Project: Brain MRI Image Segmentation Using K-Means Clustering
Module: 02 — K-Means Segmentation
Environment: R / RStudio / R Markdown

Method: Unsupervised K-Means clustering of grayscale pixel intensities
Primary dataset: `healthy.csv`
Companion dataset: `tumor.csv` (validated for the subsequent cross-image analysis)

1.1 Background and motivation

Image segmentation partitions an image into regions sharing an observed characteristic. In a grayscale MRI image, one simple representation assigns a numeric intensity to every pixel. The MIT *15.071 The Analytics Edge* Unit 6 recitation, *Seeing the Big Picture: Segmenting Images to Create Data*, uses this representation to illustrate clustering. The project moves from hierarchical clustering of a small image to K-Means for a much larger brain MRI, where computing all pairwise pixel distances is impractical.

This analysis fits **five intensity clusters** to the healthy MRI, following the course's Video 5 workflow (`set.seed(1)`, `kmeans(..., centers = 5, iter.max = 1000)`). It reconstructs the image from cluster labels, quantifies cluster sizes and within-cluster variation, and examines how intensity quantization affects the appearance of the image. The resulting clusters represent **intensity groups**, not independently identified anatomical tissues or tumors. The second MRI is reserved for the cross-image application in Module 04.

1.2 Research questions

1. Can the original healthy MRI pixel matrix be partitioned into five reproducible intensity-based groups?
2. What are the learned grayscale centroids, pixel proportions, and within-cluster dispersion?
3. How does replacing original pixel intensities with cluster-level representations change the image?
4. What do the model's numerical outputs establish, and what claims remain unsupported without anatomical labels?

1.3 Analysis roadmap

Input validation → Image representation → Five-cluster K-Means baseline → Cluster diagnostics → Segmentation visualization → Quantization error and interpretation → Reproducibility and next steps.

Project modules: 01 Data Exploration → **02 K-Means Segmentation** → 03 Model Selection → 04 Cross-Image Analysis.

1.4 Data sources and provenance

Source	Role
`Unit6_Recitation.pdf`	Background on image segmentation, clustering, computational scalability, and medical-image limitations.
`Unit6_Recitation.R`	import the healthy MRI, vectorize its pixels, fit five-cluster K-Means, and reconstruct a segmented image; applies clusters to the second image.

Source	Role
`week6_3.2.R`	Additional K-Means object inspection and subsequent elbow-plot and cross-image procedures.
`healthy.csv`	Original numeric pixel-intensity matrix used for model fitting.
`tumor.csv`	Original second MRI matrix, checked here but not used to fit or tune the baseline model.

Data scope. These CSVs encode images, not independent patients. No acquisition metadata, segmentation masks, anatomical labels, or clinically verified diagnostic outcomes accompany them. Any apparent region boundaries are based solely on pixel intensity.

2 Data Import and Integrity Checks

2.1 Locate the original CSV files

Read the data.

Hide

```
        find_data_dir <- function() {
          candidates <- c("data", "../data", ".", "..")
          valid <- vapply(candidates, function(d) {
            file.exists(file.path(d, "healthy.csv")) &&
              file.exists(file.path(d, "tumor.csv"))
          }, logical(1))
          if (!any(valid)) {
            stop("Original healthy.csv and tumor.csv not found. Place both in dat
a/.")
          }
          normalizePath(candidates[which(valid)[1]], mustWork = TRUE)
        }
        data_dir <- find_data_dir()
        cat("Original input directory:", data_dir, "\n")
```

```
## Original input directory: /Users/mac/Downloads/MIT_Analytics_Edge/Week6/3/Tumo
r
```

Code explanation. `file.exists()` requires both original input files. `normalizePath()` resolves their location. No simulated images or cached statistics are substituted for the CSVs.

2.2 Read matrices and verify intensity values

We read the CSVs with `header = FALSE`, converts each data frame into a matrix, and then flattens the matrix into a vector. Each CSV cell is one grayscale pixel; the columns are not model predictors in the conventional tabular-data sense.

Hide

```
        healthy_df <- read.csv(file.path(data_dir, "healthy.csv"),
                               header = FALSE, check.names = FALSE)
        tumor_df <- read.csv(file.path(data_dir, "tumor.csv"),
                             header = FALSE, check.names = FALSE)
        healthy_matrix <- as.matrix(healthy_df)
        tumor_matrix <- as.matrix(tumor_df)

        validate_mri <- function(x, name) {
          stopifnot(is.matrix(x), is.numeric(x), length(dim(x)) == 2L,
                    nrow(x) > 0L, ncol(x) > 0L)
          if (anyNA(x) || any(!is.finite(x))) {
            stop(name, " contains missing or non-finite intensities.")
          }
          if (any(x < 0 | x > 1)) {
            stop(name, " contains intensities outside the expected [0, 1] rang
e.")
          }
          invisible(TRUE)
        }
        validate_mri(healthy_matrix, "Healthy MRI")
        validate_mri(tumor_matrix, "Tumor MRI")

        healthy_vector <- as.vector(healthy_matrix)
        tumor_vector <- as.vector(tumor_matrix)

        # Compare values while ignoring CSV-derived row/column names.
        stopifnot(
          length(healthy_vector) == prod(dim(healthy_matrix)),
          length(tumor_vector) == prod(dim(tumor_matrix)),
          identical(as.vector(matrix(healthy_vector,
                                     nrow = nrow(healthy_matrix),
                                     ncol = ncol(healthy_matrix))), healthy_vecto
r),
          identical(as.vector(matrix(tumor_vector,
                                     nrow = nrow(tumor_matrix),
                                     ncol = ncol(tumor_matrix))), tumor_vector)
        )

        input_summary <- data.frame(
          Image = c("Healthy MRI", "Tumor MRI"),
          Rows = c(nrow(healthy_matrix), nrow(tumor_matrix)),
          Columns = c(ncol(healthy_matrix), ncol(tumor_matrix)),
          Pixels = c(length(healthy_vector), length(tumor_vector)),
          Min = c(min(healthy_vector), min(tumor_vector)),
          Max = c(max(healthy_vector), max(tumor_vector)),
          Missing = c(sum(is.na(healthy_vector)), sum(is.na(tumor_vector)))
        )
        knitr::kable(input_summary, caption = "Original MRI data validation")
```

Original MRI data validation

Image	Rows	Columns	Pixels	Min	Max	Missing
Healthy MRI	566	646	365636	0	1	0
Tumor MRI	571	512	292352	0	1	0

Output interpretation. The table reports dimensions, total pixel observations, grayscale limits, and missing-value counts computed directly from the two CSVs. The expected dimensions are 566 × 646 (365636 pixels) for the healthy MRI and 571 × 512 (292352 pixels) for the second image. All subsequent fitted quantities use `healthy_vector` only. The reconstruction check compares **pixel values**, avoiding the `identical()` failure caused by incidental matrix column names.

3 MRI Representation and Computational Rationale

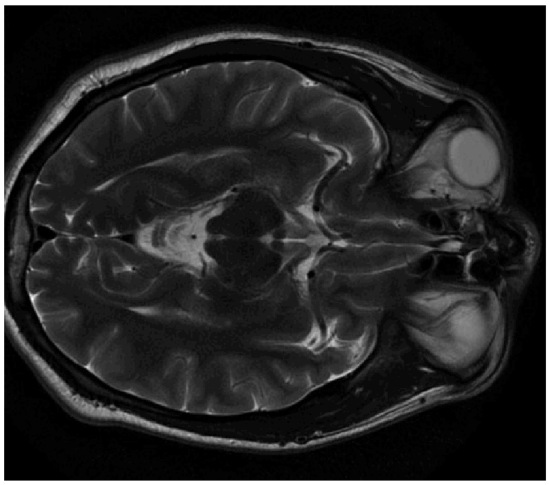
3.1 Visualize the healthy MRI

R's base `image()` treats the first matrix index as the horizontal axis. To show the CSV matrix in its usual row-down, column-right layout, we transpose it and reverse its rows before plotting. The conversion is visual only: **the clustering model uses the unmodified original intensity vector**.

Hide

```
plot_mri <- function(mat, main = "", zlim = c(0, 1),
                     col = gray(seq(0, 1, length.out = 256))) {
  stopifnot(is.matrix(mat), is.numeric(mat))
  # Transpose and reverse matrix rows for conventional image orientation.
  display_matrix <- t(mat[nrow(mat):1L, , drop = FALSE])
  image(x = seq_len(ncol(mat)), y = seq_len(nrow(mat)),
        z = display_matrix, col = col, zlim = zlim,
        xlab = "", ylab = "", axes = FALSE,
        useRaster = TRUE, main = main, asp = 1)
}
plot_mri(healthy_matrix, "Original healthy MRI — grayscale intensity")
```

Original healthy MRI — grayscale intensity



Code explanation. `plot_mri()` displays the image without altering pixel values or their order in `healthy_vector`. `zlim = c(0, 1)` uses the same grayscale range for all intensity images. The axes are suppressed because pixel coordinates are not anatomical measurements.

Output interpretation. Brighter regions correspond to larger pixel intensities; darker regions correspond to smaller ones. Visual differences in grayscale cannot by themselves identify tissue type, lesion status, or clinical pathology.

3.2 Why K-Means rather than a full hierarchical distance matrix?

Hierarchical clustering based on a full pairwise distance object requires a distance for every unordered pair of pixels. For `n` pixels, the number of distances is $n(n-1)/2$, which grows quadratically. This is why the course switches to K-Means for the MRI example; **we do not execute `dist(healthy_vector)`** .

Hide

```
n_pixels <- length(healthy_vector)
n_distances <- as.double(n_pixels) * (n_pixels - 1) / 2
estimated_gib <- n_distances * 8 / (1024^3)
cat("Healthy MRI pixels:", format(n_pixels, big.mark = ","), "\n")
```

```
## Healthy MRI pixels: 365,636
```

Hide

```
        cat("Pairwise distances:", format(n_distances, big.mark = ",", scientific
= FALSE), "\n")
```

```
## Pairwise distances: 66,844,659,430
```

Hide

```
        cat("Raw double-precision distance storage (GiB):",
            round(estimated_gib, 1), "\n")
```

```
## Raw double-precision distance storage (GiB): 498
```

Output interpretation. The estimate excludes object overhead and clustering workspace. It illustrates why constructing a full distance matrix is unsuitable for this dataset; K-Means avoids this explicit pairwise-distance storage.

4 Methodology: Five-Cluster K-Means

4.1 Objective function and assumptions

For pixel intensity $x_i \in [0, 1]$, K-Means assigns each pixel to one of k clusters and minimizes the within-cluster sum of squared deviations:

$$\mathrm{WCSS} = \sum_{j=1}^{k} \sum_{i:c_i=j} (x_i - \mu_j)^2,$$

where c_i is the cluster assignment and μ_j is the mean intensity of cluster j. With only one feature, the learned centroids are **grayscale levels**, not anatomical locations. Pixels far apart in the image may belong to the same cluster if their intensities are similar.

4.2 Fit the course baseline (k = 5)

The model below follows the core parameters in the supplied R scripts: `set.seed(1)`, five clusters, and `iter.max = 1000`. We retain R's default `nstart = 1` for the course baseline; repeated initializations and sensitivity analyses belong to Module 03.

Hide

```
k <- 5L
set.seed(1)
KMC <- kmeans(healthy_vector, centers = k, iter.max = 1000)

stopifnot(
  length(KMC$cluster) == length(healthy_vector),
  length(KMC$centers) == k,
  sum(KMC$size) == length(healthy_vector),
  all(KMC$cluster %in% seq_len(k)),
  all(is.finite(KMC$centers))
)

cat("Requested clusters:", k, "\n")
```

```
## Requested clusters: 5
```

Hide

```
cat("Actual clusters:", length(KMC$size), "\n")
```

```
## Actual clusters: 5
```

Hide

```
cat("Iterations:", KMC$iter, "\n")
```

```
## Iterations: 3
```

Hide

```
cat("Total within-cluster sum of squares:", signif(KMC$tot.withinss, 7),
"\n")
```

```
## Total within-cluster sum of squares: 302.1291
```

Hide

```
cat("Between-cluster sum of squares:", signif(KMC$betweenss, 7), "\n")
```

```
## Between-cluster sum of squares: 5472.814
```

Code explanation. `KMC$cluster` contains one assignment per pixel; `KMC$centers` stores the five learned mean intensities; `KMC$size` gives the number of pixels assigned to each cluster; `KMC$withinss` gives the squared-error contribution of each cluster. `KMC$tot.withinss` is their sum. The seed fixes the random initialization in a given R environment, but it does not guarantee identical labels across all software versions or implementations.

Output interpretation. The model partitioned 365,636 pixel observations into 5 nonempty intensity groups in 3 iteration(s). Its total within-cluster sum of squares is 302.129. This measures grayscale compactness, **not** segmentation accuracy against medical ground truth.

5 Cluster Characteristics and Diagnostics

5.1 Centroids, sizes, and dispersion

Cluster IDs are arbitrary: cluster 1 is not inherently darker than cluster 2. To make the results interpretable, the next table sorts clusters by **ascending learned intensity** while preserving their original IDs.

Hide

```
        cluster_table <- data.frame(
          Original_Cluster_ID = seq_len(k),
          Centroid = as.numeric(KMC$centers),
          Pixels = as.integer(KMC$size),
          Proportion = as.numeric(KMC$size) / length(healthy_vector),
          Within_SS = as.numeric(KMC$withinss)
        )
        cluster_table <- cluster_table[order(cluster_table$Centroid), ]
        cluster_table$Intensity_Rank <- seq_len(k)
        cluster_table <- cluster_table[, c("Intensity_Rank", "Original_Cluster_I
D",
                                           "Centroid", "Pixels", "Proportion", "W
ithin_SS")]
        knitr::kable(cluster_table, digits = 5,
                     caption = "K-Means cluster diagnostics ordered from darkest
to brightest")
```

K-Means cluster diagnostics ordered from darkest to brightest

	Intensity_Rank	Original_Cluster_ID	Centroid	Pixels	Proportion	Within_SS
5	1	5	0.02189	139963	0.38279	53.30527
1	2	1	0.11656	114480	0.31310	65.91078
2	3	2	0.20136	65557	0.17930	54.41617
4	4	4	0.32991	27338	0.07477	47.37668
3	5	3	0.49139	18298	0.05004	81.12020

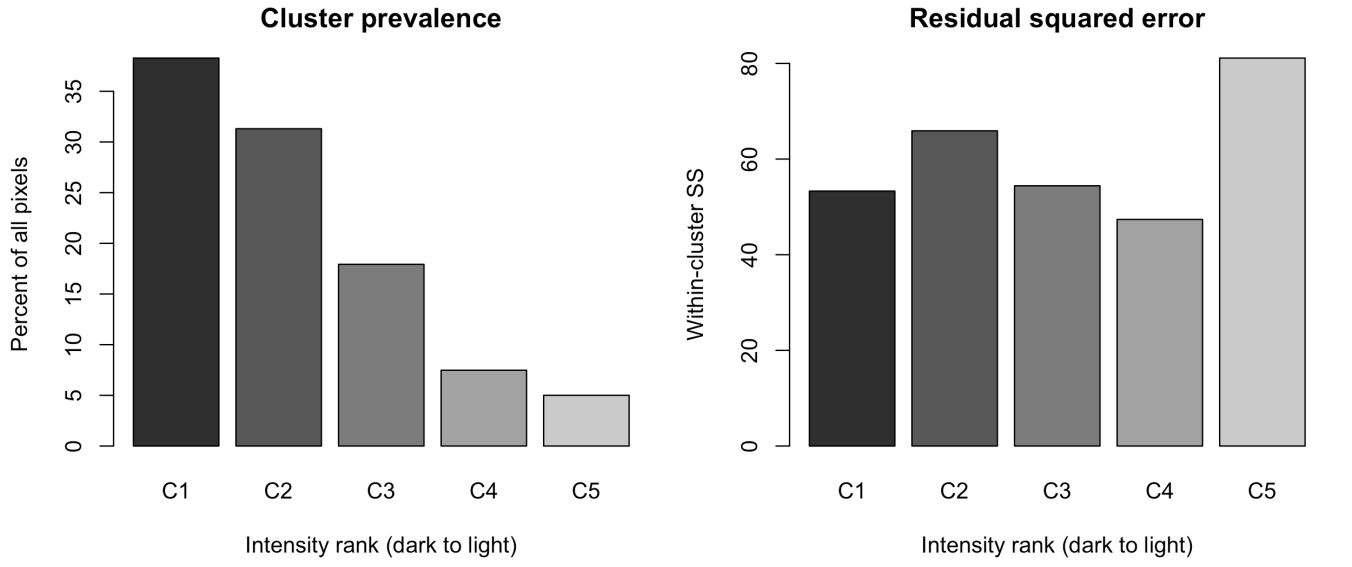
Code explanation. The intensity rank is for presentation only; it does not retrain the model or change its assignments. Pixel proportions use the total number of pixels, **including dark background pixels**, as the denominator.

Output interpretation. The learned centroids range from 0.0219 to 0.4914 on the original 0–1 grayscale scale. The largest cluster contains 139,963 pixels (38.3% of the image). Cluster size describes prevalence of an intensity band, not the area of a verified anatomical tissue.

5.2 Cluster sizes and within-cluster variation

Hide

```
        op <- par(mfrow = c(1, 2), mar = c(5, 4.2, 3, 1))
        barplot(cluster_table$Proportion * 100,
                names.arg = paste0("C", cluster_table$Intensity_Rank),
                col = gray(seq(0.2, 0.8, length.out = k)),
                xlab = "Intensity rank (dark to light)", ylab = "Percent of all p
ixels",
                main = "Cluster prevalence")
        barplot(cluster_table$Within_SS,
                names.arg = paste0("C", cluster_table$Intensity_Rank),
                col = gray(seq(0.2, 0.8, length.out = k)),
                xlab = "Intensity rank (dark to light)", ylab = "Within-cluster S
S",
                main = "Residual squared error")
```



Cluster size and within-cluster sum of squares, ordered by centroid intensity.

Hide

```
        par(op)
```

Output interpretation. A large cluster indicates many pixels with similar intensity. A high within-cluster sum of squares can reflect both the number of pixels and the range of intensities within that cluster; it should not be interpreted as clinical uncertainty. These charts are descriptive diagnostics for the fitted five-cluster solution.

5.3 Verify centroid and WCSS calculations

The following independent calculations check that every centroid equals the mean intensity of its assigned pixels, and that the squared residuals reproduce the model's WCSS.

Hide

```
recomputed_centers <- tapply(healthy_vector, KMC$cluster, mean)
recomputed_wcss <- sum((healthy_vector -
                        as.numeric(KMC$centers)[KMC$cluster])^2)
stopifnot(
  isTRUE(all.equal(as.numeric(recomputed_centers),
                   as.numeric(KMC$centers), tolerance = 1e-8)),
  isTRUE(all.equal(recomputed_wcss, KMC$tot.withinss,
                   tolerance = 1e-8)),
  isTRUE(all.equal(sum(KMC$withinss), KMC$tot.withinss,
                   tolerance = 1e-8))
)
cat("Centroid means match: TRUE\n")
```

```
## Centroid means match: TRUE
```

Hide

```
cat("Recomputed WCSS matches: TRUE\n")
```

```
## Recomputed WCSS matches: TRUE
```

Hide

```
cat("Cluster WCSS sums to total: TRUE\n")
```

```
## Cluster WCSS sums to total: TRUE
```

Output interpretation. Passing all three checks confirms internal arithmetic consistency for the fitted model. These are reproducibility checks, not an external validation of medical segmentation quality.

6 Visualizing the Segmented MRI

6.1 Construct the cluster-label image

The model assigns clusters to a flattened vector in R's **column-major order**. Constructing with the original row and column counts preserves the pixel-to-location mapping. We sort the clusters by their centroid intensity for display so darker intensity groups appear darker in the figure.

Hide

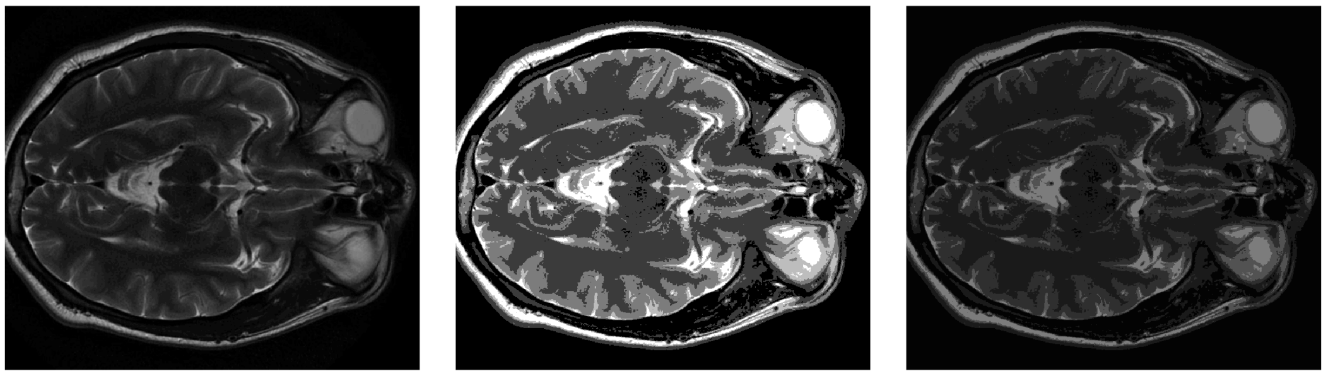
```
        # Convert original arbitrary cluster IDs into ordered grayscale ranks.
        rank_by_original_id <- integer(k)
        rank_by_original_id[cluster_table$Original_Cluster_ID] <- seq_len(k)
        ordered_cluster_vector <- rank_by_original_id[KMC$cluster]
        cluster_matrix <- matrix(ordered_cluster_vector,
                                 nrow = nrow(healthy_matrix),
                                 ncol = ncol(healthy_matrix))

        # Replace every original pixel with its assigned centroid intensity.
        centroid_vector <- as.numeric(KMC$centers)[KMC$cluster]
        centroid_matrix <- matrix(centroid_vector,
                                  nrow = nrow(healthy_matrix),
                                  ncol = ncol(healthy_matrix))

        stopifnot(
          identical(dim(cluster_matrix), dim(healthy_matrix)),
          identical(dim(centroid_matrix), dim(healthy_matrix)),
          length(unique(as.vector(cluster_matrix))) == k
        )

        plot_mri <- function(mat, main = "", col = gray(seq(0, 1, length.out = 25
6)),
                             zlim = c(0, 1)) {
          stopifnot(is.matrix(mat), is.numeric(mat))
          display_matrix <- t(mat[nrow(mat):1L, , drop = FALSE])
          image(x = seq_len(ncol(mat)), y = seq_len(nrow(mat)),
                z = display_matrix, col = col, zlim = zlim,
                axes = FALSE, xlab = "", ylab = "", useRaster = TRUE,
                main = main, asp = 1)
        }

        op <- par(mfrow = c(1, 3), mar = c(2, 1, 3, 1))
        plot_mri(healthy_matrix, "Original grayscale")
        plot_mri(cluster_matrix, "Cluster labels (5 levels)",
                 col = gray(seq(0, 1, length.out = k)), zlim = c(1, k))
        plot_mri(centroid_matrix, "Centroid reconstruction")
```



Original healthy MRI, five intensity-cluster labels, and centroid-based grayscale reconstruction.

Hide

```
    par(op)
```

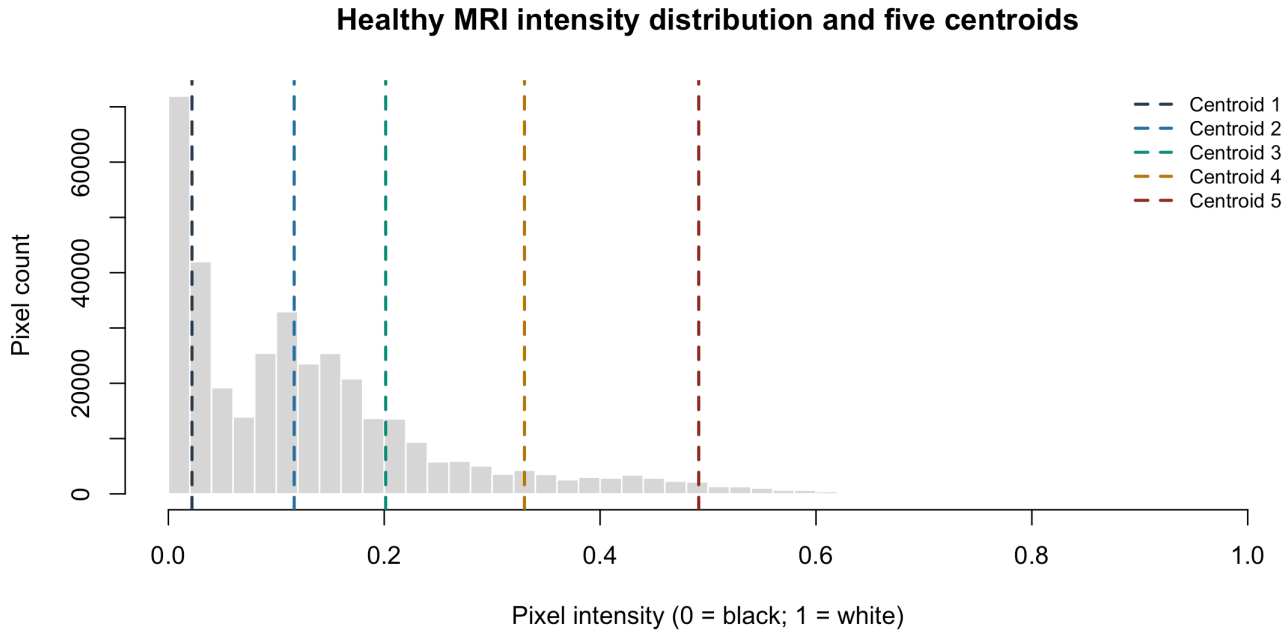
Code explanation. The middle panel displays categorical cluster membership, ordered from low to high centroid intensity. The right panel displays each pixel at the actual numerical intensity of its assigned centroid, rather than using arbitrary categorical colors. Both views are useful: the label map emphasizes the partition, while the centroid reconstruction permits direct comparison with the original grayscale image.

Output interpretation. The five-level segmentation retains coarse intensity structures while discarding variation **within** each group. Boundaries can appear abrupt because neighboring pixels with slightly different intensities may receive different labels. Conversely, disconnected areas can receive the same label. The plot does not identify tumor tissue or establish anatomical accuracy.

6.2 Intensity distribution and centroid positions

Hide

```
hist(healthy_vector, breaks = seq(0, 1, by = 0.02),
     col = "gray85", border = "white",
     main = "Healthy MRI intensity distribution and five centroids",
     xlab = "Pixel intensity (0 = black; 1 = white)",
     ylab = "Pixel count", xlim = c(0, 1))
abline(v = sort(as.numeric(KMC$centers)),
       col = c("#2c3e50", "#2471a3", "#148f77", "#b9770e", "#922b21"),
       lwd = 2, lty = 2)
legend("topright", legend = paste0("Centroid ", seq_len(k)),
       col = c("#2c3e50", "#2471a3", "#148f77", "#b9770e", "#922b21"),
       lty = 2, lwd = 2, cex = 0.8, bty = "n")
```



Healthy MRI pixel-intensity histogram with learned K-Means centroids.

Output interpretation. Each vertical line marks a fitted centroid; the histogram shows how frequently the corresponding pixel intensities occur. K-Means places centroids to reduce squared reconstruction error, so their locations are influenced by the full intensity distribution. A centroid is not necessarily a distinct biological tissue class.

7 Quantifying Segmentation Error

7.1 Pixel-level grayscale reconstruction error

Since each pixel is replaced with its cluster centroid, K-Means can be interpreted as a **five-level grayscale quantizer**. Its mean squared error (MSE) is total WCSS divided by the number of pixels. Root mean squared error (RMSE) expresses the typical deviation on the original intensity scale.

Hide

```
        residuals <- healthy_vector - centroid_vector
        mse <- mean(residuals^2)
        rmse <- sqrt(mse)
        mae <- mean(abs(residuals))
        variance_total <- sum((healthy_vector - mean(healthy_vector))^2)
        variance_explained <- 1 - KMC$tot.withinss / variance_total
        metrics <- data.frame(
          Metric = c("MSE", "RMSE", "MAE", "Variance explained (descriptive)"),
          Value = c(mse, rmse, mae, variance_explained)
        )
        knitr::kable(metrics, digits = 6,
                     caption = "Grayscale reconstruction diagnostics for the five
-cluster model")
```

Grayscale reconstruction diagnostics for the five-cluster model

Metric	Value
MSE	0.000826
RMSE	0.028746
MAE	0.022047
Variance explained (descriptive)	0.947683

Hide

```
        stopifnot(isTRUE(all.equal(mse * length(healthy_vector), KMC$tot.withins
s,
                                   tolerance = 1e-8)))
```

Code explanation. The residual for pixel i is $x_i - \mu_{c_i}$. The `variance explained` value compares within-cluster squared error with total squared variation around the image's global mean. It is an in-sample descriptive measure of intensity compression, **not predictive accuracy** and not the fraction of clinically relevant tissue correctly segmented.

Output interpretation. The fitted five-level approximation has RMSE 0.02875 and MAE 0.02205 on a 0–1 intensity scale. The descriptive proportion of grayscale variation represented by the cluster centroids is 94.77%. These metrics quantify numerical approximation of the **same training image**, so they cannot establish out-of-image generalization.

8 Discussion

8.1 What the model demonstrates

The K-Means baseline groups 365,636 healthy MRI pixels using **only one feature per pixel: grayscale intensity**. Five learned centroids summarize the image's intensity distribution, and cluster labels can be reshaped into a spatial image without losing the original pixel indexing. The fitted model provides a reproducible example of unsupervised image segmentation and image quantization.

The centroid and cluster-size diagnostics characterize how grayscale values are distributed among the five groups. The reconstruction-error metrics describe the cost of replacing a continuous intensity scale with five representative levels. Neither classifies pixels as healthy versus abnormal, nor verifies anatomical boundaries.

8.2 Important limitations

- **No ground-truth segmentation masks.** Dice, Intersection over Union, sensitivity, specificity, and clinical accuracy cannot be estimated from the supplied files.
- **No spatial features.** Adjacent pixels are not explicitly encouraged to share labels; distant pixels with similar grayscale intensities can be assigned together.
- **Background influences clustering.** Dark non-anatomical background pixels are included in the fitted centroids and cluster proportions.
- **Five clusters are a baseline, not an optimum.** A systematic evaluation of `k`, random initializations, and segmentation sensitivity is undertaken in Module 03.
- **No clinical interpretation.** Intensity clusters should not be relabeled as tumor, edema, gray matter, or white matter without appropriate validation.
- **One image for model development.** The second MRI has different dimensions and is not registered to the first. Out-of-sample cluster assignment and its limitations are considered in Module 04.

9 Conclusion and Next Steps

A five-cluster K-Means model was fitted to the healthy MRI intensity values. The segmentation creates an interpretable, lower-complexity representation of grayscale variation, with centroids, pixel proportions, and errors computed directly from the input CSV. The result is an **intensity-based partition**, not a medical diagnosis.

Module 03 — Model Selection will examine the sensitivity of segmentation to the number of clusters and initialization choices, extending the elbow-plot logic in `week6_3.2.R`. **Module 04 — Cross-Image Analysis** will assign pixels from `tumor.csv` to the healthy-image centroids and compare the resulting intensity-cluster distributions without claiming tumor detection.

10 Reproducibility

The model uses base R `stats::kmeans()` and `graphics` functions; `knitr` and `rmarkdown` are required for document rendering. All calculations are regenerated from the original CSVs at Knit time. The random seed is set immediately before model fitting. No package installation is performed inside the analysis document.

Hide

```
sessionInfo()
```

```
## R version 4.2.1 (2022-06-23)
## Platform: x86_64-apple-darwin17.0 (64-bit)
## Running under: macOS Big Sur ... 10.16
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.2/Resources/lib/libRblas.0.
dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.2/Resources/lib/libRlapack.
dylib
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## loaded via a namespace (and not attached):
##  [1] digest_0.6.33     R6_2.5.1          lifecycle_1.0.4   jsonlite_1.8.8
##  [5] evaluate_0.23     highr_0.10        cachem_1.0.8      rlang_1.1.2
##  [9] cli_3.6.2         rstudioapi_0.15.0 jquerylib_0.1.4   bslib_0.6.1
## [13] rmarkdown_2.25    tools_4.2.1       xfun_0.41         yaml_2.3.8
## [17] fastmap_1.1.1     compiler_4.2.1    htmltools_0.5.7   knitr_1.45
## [21] sass_0.4.8
```

11 References

1. Massachusetts Institute of Technology, *15.071x The Analytics Edge*, Unit 6 Recitation: *Seeing the Big Picture: Segmenting Images to Create Data*, lecture slides (`Unit6_Recitation.pdf`).
2. MIT course R implementation, `Unit6_Recitation.R` , Videos 4–6.
3. Supporting R analysis, `week6_3.2.R` (K-Means diagnostics, elbow analysis, and cross-image assignment).
4. Original image-intensity data: `healthy.csv` and `tumor.csv` (supplied course data).